Projet de fin de module - Deep Learning

Conception, implémentation, comparaison et analyse critique de modèles de deep learning pour données tabulaires, images et séquences

Étudiant	Oussama Oulkaid
Module	Deep Learning
Année universitaire	2025-2026
Nature du travail	Projet individuel intégrateur
Environnement cible	ASUS VivoBook S14, 16 GB RAM, exécution CPU possible
Framework	PyTorch

Note méthodologique : les métriques numériques finales doivent être obtenues en exécutant les scripts fournis. Aucun résultat expérimental n'est inventé dans ce document. Les tableaux de résultats sont conçus pour être remplis avec les fichiers générés automatiquement dans le dossier results.

Sommaire

1. 1. Introduction générale
2. 2. Méthodologie générale du projet
3. 3. Partie I - MLP et données tabulaires
4. 4. Partie II - CNN et vision par ordinateur
5. 5. Partie III - RNN, LSTM, GRU et Seq2Seq
6. 6. Discussion transversale finale
7. 7. Conclusion générale
8. 8. Guide de reproduction expérimentale
9. 9. Annexes

1. Introduction générale

Le deep learning constitue aujourd'hui l'un des paradigmes les plus importants de l'apprentissage automatique supervisé. Cependant, son efficacité ne repose pas uniquement sur la profondeur des modèles, mais surtout sur l'adéquation entre l'architecture choisie et la structure des données étudiées. Une donnée tabulaire, une image et une séquence textuelle ne possèdent ni la même géométrie, ni les mêmes dépendances internes, ni les mêmes contraintes de représentation.

Ce projet a pour objectif d'étudier, d'implémenter et de comparer trois familles de modèles : le perceptron multicouche pour les données tabulaires, le réseau de neurones convolutionnel pour les images, et les architectures récurrentes pour les séquences. L'enjeu principal est de montrer que le choix du modèle doit être justifié par la nature statistique des données, et non par une simple préférence technique.

La démarche adoptée combine une étude théorique, une implémentation en PyTorch, une expérimentation comparative et une analyse critique. Chaque partie suit une logique similaire : préparation des données, définition de l'architecture, entraînement, évaluation, interprétation et discussion des limites.

2. Méthodologie générale du projet

Le projet est organisé en trois blocs expérimentaux indépendants mais cohérents. Le premier bloc traite la classification supervisée sur données tabulaires à l'aide d'un MLP. Le deuxième bloc traite la classification d'images à l'aide d'un CNN et met l'accent sur les opérations convolutionnelles. Le troisième bloc traite les données séquentielles au moyen de RNN, LSTM, GRU et d'un mini système Seq2Seq.

Tous les scripts sont conçus pour fonctionner sur CPU, ce qui rend le projet exécutable sur un ordinateur portable standard. La détection du device est néanmoins intégrée afin de respecter les bonnes pratiques PyTorch et de permettre l'utilisation automatique d'un GPU si disponible.

Partie	Nature des données	Architecture principale	Objectif expérimental
I	Tabulaire	MLP	Comparer implémentations et initialisations
II	Images	CNN	Comparer MLP/CNN et choix convolutionnels
III	Séquences textuelles	RNN, LSTM, GRU, Seq2Seq	Comparer mémoire, stabilité et décodage

Les métriques utilisées sont adaptées au type de tâche : accuracy, precision, recall, F1-score et matrice de confusion pour les tâches de classification ; score BLEU simplifié pour la démonstration Seq2Seq. Les courbes d'apprentissage permettent d'observer la convergence, l'écart train/validation et les signes éventuels de surapprentissage.

3. Partie I - MLP et données tabulaires

3.1 Objectif et dataset

La première partie étudie un perceptron multicouche appliqué à un problème de classification tabulaire. Le dataset recommandé dans l'implémentation est Breast Cancer Wisconsin, accessible via scikit-learn. Il contient des caractéristiques numériques décrivant des cellules tumorales et une classe cible binaire.

Ce choix est pertinent pour un MLP car les données sont structurées sous forme de vecteurs de caractéristiques indépendantes, sans organisation spatiale explicite comme dans une image. La normalisation est indispensable afin d'éviter qu'une variable de grande amplitude domine artificiellement l'apprentissage.

3.2 Concepts PyTorch fondamentaux

Un modèle PyTorch hérite généralement de nn.Module. Cette classe fournit la gestion automatique des paramètres entraînables, l'intégration avec l'autograd, la sauvegarde via state_dict et le passage entre CPU et GPU. La méthode forward définit la propagation avant, tandis que la rétropropagation est déclenchée par loss.backward().

Le state_dict contient l'ensemble des poids et biais du modèle. Il permet de sauvegarder uniquement les paramètres appris sans sérialiser l'objet complet. La fonction named_parameters() est utilisée pour inspecter explicitement les tenseurs entraînables, leur nom, leur dimension et leur statut requires_grad.

Concept	Rôle dans le projet
nn.Module	Base de définition d'un modèle personnalisé
nn.Sequential	Construction rapide d'un empilement de couches
Gradient	Quantifie l'effet des paramètres sur la perte
state_dict	Sauvegarde et rechargement des poids
device	Cohérence entre modèle et données sur CPU/GPU
Backpropagation	Calcul automatique des gradients par autograd

3.3 Préparation des données

La préparation suit quatre étapes : chargement du dataset, séparation en ensembles d'apprentissage, validation et test, normalisation par StandardScaler, puis conversion en tenseurs PyTorch. Le scaler est ajusté uniquement sur les données d'apprentissage afin d'éviter toute fuite d'information depuis les ensembles de validation ou de test.

L'utilisation d'un DataLoader permet de gérer les mini-lots, le mélange des données d'apprentissage et la reproductibilité des expérimentations. Cette étape est essentielle car un entraînement par batch complet serait moins flexible et moins représentatif des pratiques modernes en deep learning.

3.4 Architectures implémentées

Deux versions du MLP sont implémentées : une version construite avec nn.Sequential et une version construite sous forme de classe personnalisée. Les deux modèles possèdent la même structure afin que la comparaison porte sur la méthode d'implémentation plutôt que sur la capacité du réseau.

Linear(30 -> 64) -> ReLU -> Linear(64 -> 32) -> ReLU -> Linear(32 -> 2)

3.5 Initialisation des poids

Trois stratégies d'initialisation sont testées : gaussienne, constante et Xavier. L'initialisation constante est volontairement défavorable car elle tend à créer une symétrie entre neurones. L'initialisation gaussienne peut fonctionner mais dépend fortement de l'écart-type choisi. L'initialisation Xavier est théoriquement plus stable pour conserver une variance raisonnable des activations et des gradients.

Initialisation	Principe	Effet attendu
Gaussienne	Poids tirés selon une loi normale	Résultat sensible à l'écart-type
Constante	Même valeur pour plusieurs poids	Risque de symétrie et faible diversité
Xavier	Variance contrôlée selon les dimensions	Propagation plus stable des gradients

3.6 Tableau de résultats à compléter

Modèle	Initialisation	Accuracy	Precision	Recall	F1-score	Commentaire
Sequential	Gaussienne	à générer	à générer	à générer	à générer	Analyser convergence
Sequential	Constante	à générer	à générer	à générer	à générer	Comparer stabilité
Sequential	Xavier	à générer	à générer	à générer	à générer	Vérifier gain attendu
Classe personnalisée	Gaussienne	à générer	à générer	à générer	à générer	Comparer à Sequential
Classe personnalisée	Constante	à générer	à générer	à générer	à générer	Identifier limites
Classe personnalisée	Xavier	à générer	à générer	à générer	à générer	Meilleur candidat attendu

3.7 Analyse critique attendue

Un MLP bien paramétré peut être pertinent pour les données tabulaires lorsque les variables explicatives sont déjà extraites sous forme numérique et que la relation entre les variables et la cible peut être approximée par des combinaisons non linéaires. Sa force réside dans sa flexibilité et sa simplicité d'implémentation.

Ses limites apparaissent lorsque la structure des données contient des dépendances géométriques ou temporelles non représentées dans le vecteur d'entrée. Le MLP ne possède pas de mécanisme explicite de partage de poids, de localité spatiale ou de mémoire séquentielle. Il dépend donc fortement de la qualité des features et du prétraitement.

4. Partie II - CNN et vision par ordinateur

4.1 Pourquoi un MLP est limité pour les images

Une image n'est pas seulement un vecteur de pixels : elle possède une structure spatiale. Deux pixels voisins ont généralement une relation locale forte. En aplatissant l'image pour l'envoyer dans un MLP, on perd cette structure et on augmente fortement le nombre de paramètres. Le modèle doit alors réapprendre implicitement des relations spatiales que le CNN encode naturellement.

Un CNN exploite trois idées fondamentales : la localité des récepteurs, le partage des poids et la hiérarchie des représentations. Les premières couches détectent souvent des motifs simples comme des bords, tandis que les couches profondes combinent ces motifs en formes plus abstraites.

4.2 Opérations convolutionnelles et calculs dimensionnels

La convolution utilisée dans les bibliothèques de deep learning correspond généralement à une corrélation croisée. Pour une entrée de taille W, un noyau de taille K, un padding P et un stride S, la taille de sortie est donnée par : floor((W - K + 2P) / S) + 1.

Exemple : pour Fashion-MNIST, une image de 28 x 28, un noyau 3 x 3, un stride de 1 et aucun padding donne une sortie de 26 x 26. Un max-pooling 2 x 2 avec stride 2 réduit ensuite la dimension à 13 x 13.

Paramètre	Effet principal
Padding	Conserve ou augmente la taille spatiale en ajoutant des bords
Stride	Réduit la résolution spatiale et accélère le calcul
Pooling	Réduit la dimension et rend la représentation plus robuste
Nombre de filtres	Augmente la capacité de détection de motifs
Convolution 1 x 1	Combine les canaux et ajuste la profondeur

4.3 Modèle CNN

Le modèle retenu est une variante légère inspirée de LeNet. Il contient deux blocs convolutionnels suivis d'un classifieur fully connected. Cette architecture est suffisamment simple pour être entraînée sur CPU, tout en permettant d'observer clairement l'intérêt des convolutions par rapport à un MLP.

Conv2d -> ReLU -> Pooling -> Conv2d -> ReLU -> Pooling -> Flatten -> Linear -> ReLU -> Linear

4.4 Plan expérimental

Les expériences comparent d'abord un MLP baseline et un CNN baseline sur le même dataset d'images. Ensuite, plusieurs variantes CNN sont étudiées : pooling moyen au lieu de max-pooling, ajout de padding, réduction du nombre de filtres, et ajout d'une convolution 1 x 1.

Expérience	Variable testée	Interprétation attendue
MLP baseline	Image aplatie	Mesurer la limite d'un modèle sans structure spatiale
CNN baseline	Convolution + max-pooling	Évaluer le gain lié à la localité
Average pooling	Type de pooling	Comparer lissage et sélection du maximum
Padding	Conservation des bords	Observer l'effet sur les informations périphériques
Moins de filtres	Capacité du modèle	Tester le compromis performance/coût
Conv 1 x 1	Mélange des canaux	Observer le gain ou la redondance

4.5 Tableau de résultats à compléter

Modèle	Accuracy	Precision	Recall	F1-score	Paramètres	Interprétation
MLP baseline	à générer	à générer	à générer	à générer	à générer	Comparer au CNN
CNN baseline	à générer	à générer	à générer	à générer	à générer	Gain spatial attendu
CNN average pooling	à générer	à générer	à générer	à générer	à générer	Impact du pooling
CNN padding	à générer	à générer	à générer	à générer	à générer	Effet des bords
CNN moins de filtres	à générer	à générer	à générer	à générer	à générer	Compromis capacité/coût
CNN conv 1 x 1	à générer	à générer	à générer	à générer	à générer	Mélange des canaux

4.6 Analyse critique attendue

Un CNN est généralement plus pertinent qu'un MLP pour les images car il respecte la géométrie locale des pixels et réduit le nombre de paramètres grâce au partage des poids. Cette contrainte inductive améliore souvent la généralisation, surtout lorsque les motifs visuels se répètent dans différentes zones de l'image.

Le padding peut préserver des informations situées aux bords, mais il ajoute aussi des valeurs artificielles. Le stride accélère le calcul mais peut supprimer des détails fins. Le pooling réduit la dimension et peut rendre le modèle plus robuste, mais une réduction trop agressive diminue l'information disponible. La profondeur et le nombre de filtres augmentent la capacité du modèle mais accroissent le risque de surapprentissage et le coût de calcul.

5. Partie III - RNN, LSTM, GRU et Seq2Seq

5.1 Objectif probabiliste des modèles de séquence

Une séquence textuelle est ordonnée : le sens d'un token dépend souvent des tokens précédents. Un modèle de langage cherche à factoriser la probabilité d'une séquence selon la règle de chaîne : P(w1, ..., wT) = produit de P(wt | w< t). Cette formulation justifie l'utilisation de modèles capables de conserver un état caché dépendant du passé.

Dans ce projet, les architectures RNN, LSTM et GRU sont comparées sur une tâche de classification textuelle. Un mini système Seq2Seq est également fourni pour illustrer l'architecture encodeur-décodeur, le teacher forcing, le décodage glouton et le beam search.

5.2 Préparation textuelle

Le prétraitement textuel comprend la tokenisation, la construction d'un vocabulaire, l'ajout de tokens spéciaux, le padding à longueur fixe et la création de mini-lots. Les tokens <pad> et <unk> permettent respectivement de compléter les séquences courtes et de gérer les mots absents du vocabulaire.

Élément	Rôle
Tokenisation	Transformer le texte brut en unités lexicales
Vocabulaire	Associer chaque token à un identifiant numérique
Padding	Obtenir des tenseurs de longueur homogène
Embedding	Projeter les tokens dans un espace vectoriel dense
Gradient clipping	Limiter l'explosion des gradients pendant le BPTT

5.3 RNN, LSTM et GRU

Le RNN simple met à jour un état caché à chaque pas temporel. Son principal défaut est sa difficulté à conserver des dépendances longues à cause du phénomène de vanishing gradient. Le LSTM introduit une cellule mémoire et des portes d'entrée, d'oubli et de sortie pour mieux contrôler la circulation de l'information. Le GRU simplifie cette logique avec moins de paramètres, ce qui peut le rendre plus rapide tout en conservant une bonne capacité de mémorisation.

Modèle	Avantage	Limite
RNN simple	Architecture simple et peu coûteuse	Mémoire longue faible
LSTM	Bonne gestion des dépendances longues	Plus de paramètres et coût supérieur
GRU	Compromis efficacité/stabilité	Moins expressif que LSTM dans certains cas
Seq2Seq	Adapté à la génération et traduction	Sensibilité au décodage et aux données

5.4 Seq2Seq, teacher forcing et décodage

Un modèle Seq2Seq contient un encodeur qui résume la séquence source dans un état latent, puis un décodeur qui génère la séquence cible token par token. Pendant l'apprentissage, le teacher

forcing consiste à fournir parfois le vrai token précédent au décodeur au lieu du token prédit, ce qui stabilise et accélère l'entraînement.

Deux stratégies de décodage sont comparées : le greedy decoding, qui choisit à chaque étape le token le plus probable, et le beam search, qui conserve plusieurs hypothèses candidates. Le beam search est généralement plus robuste car il réduit le risque de prendre une mauvaise décision irréversible très tôt dans la génération.

5.5 Tableaux de résultats à compléter

Modèle	Accuracy	Precision	Recall	F1-score	Paramètres	Commentaire
RNN	à générer	à générer	à générer	à générer	à générer	Référence simple
LSTM	à générer	à générer	à générer	à générer	à générer	Mémoire longue
GRU	à générer	à générer	à générer	à générer	à générer	Compromis efficacité

Source	Référence	Greedy	Beam search	BLEU-1
à générer	à générer	à générer	à générer	à générer

5.6 Analyse critique attendue

Les architectures récurrentes sont adaptées aux séquences parce qu'elles traitent explicitement l'ordre des éléments. Le passage d'un RNN simple vers un LSTM ou un GRU se justifie par la nécessité de mieux contrôler la mémoire et les gradients. Le passage vers Seq2Seq se justifie lorsque la sortie est elle-même une séquence, comme en traduction ou en génération textuelle.

Les limites observables concernent le coût séquentiel de l'entraînement, la sensibilité à la longueur maximale, la taille du vocabulaire et la qualité du corpus. Contrairement aux CNN, les RNN ne parallélisent pas naturellement tous les pas temporels, ce qui peut ralentir l'entraînement sur de longues séquences.

6. Discussion transversale finale

La problématique centrale du projet est la suivante : comment le deep learning adapte-t-il ses architectures à la structure des données tabulaires, visuelles et séquentielles ? Les trois parties montrent que le même paradigme supervisé se décline différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données.

Pour les données tabulaires, le MLP reçoit directement un vecteur de caractéristiques. Sa capacité vient de la composition de transformations linéaires et non linéaires. Pour les images, le CNN introduit une contrainte inductive spatiale : les filtres se déplacent localement et partagent leurs poids sur toute l'image. Pour les séquences, les modèles récurrents introduisent un état caché qui évolue dans le temps et permet d'encoder une dépendance d'ordre.

Type de données	Structure dominante	Architecture adaptée	Justification
Tabulaire	Vecteur de variables	MLP	Approximation non linéaire de relations entre features
Image	Grille spatiale	CNN	Localité, partage des poids, hiérarchie visuelle
Séquence	Ordre temporel	RNN/LSTM/GRU	Mémoire et dépendances entre tokens
Séquence vers séquence	Entrée et sortie ordonnées	Seq2Seq	Encodage global puis génération progressive

Ainsi, l'efficacité du deep learning ne provient pas seulement de l'optimisation par gradient, mais de la combinaison entre apprentissage statistique et hypothèses architecturales pertinentes. Une architecture mal adaptée peut nécessiter davantage de données, davantage de paramètres et produire une généralisation inférieure.

7. Conclusion générale

Ce projet montre que le choix d'une architecture de deep learning doit être guidé par la nature des données. Le MLP est adapté aux données tabulaires normalisées, le CNN exploite efficacement la structure spatiale des images, et les architectures récurrentes permettent de traiter des séquences ordonnées. Les expérimentations proposées permettent de relier les concepts théoriques aux résultats mesurables.

La qualité du travail final dépendra de l'exécution rigoureuse des scripts, de l'insertion des métriques réelles dans les tableaux, de l'interprétation des courbes et de la discussion honnête des limites. Une simple accumulation de résultats n'est pas suffisante : chaque figure et chaque métrique doivent être reliées à une hypothèse expérimentale et à une conclusion scientifique.

8. Guide de reproduction expérimentale

Les commandes suivantes permettent de reproduire les résultats. Il est conseillé d'exécuter les parties dans l'ordre afin de valider progressivement l'environnement et de limiter les erreurs de dépendances.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

python part1_mlp/train_mlp.py --epochs 30 --batch-size 32
python part2_cnn/train_cnn.py --epochs 8 --batch-size 64 --max-train 12000 --max-test 2000
python part3_rnn/train_rnn_text.py --epochs 6 --batch-size 32 --max-vocab 12000 --max-len 120
python part3_rnn/tiny_seq2seq.py --epochs 300
python report/generate_report_from_results.py
```

Si Fashion-MNIST ne se télécharge pas, la commande alternative suivante utilise le dataset digits de scikit-learn :

```
python part2_cnn/train_cnn.py --dataset digits --epochs 12 --batch-size 64
```

9. Annexes

9.1 Checklist de qualité

- Le rapport contient une introduction, une méthodologie, des résultats, une interprétation, des limites et une conclusion.
- Chaque tableau de métriques est interprété et non simplement affiché.
- Les courbes d'apprentissage sont commentées en lien avec la convergence et le surapprentissage.
- Les matrices de confusion sont interprétées par classe.
- Les choix d'architecture sont reliés à la structure des données.
- Le code est commenté et exécutable depuis la racine du projet.

9.2 Limites générales

Les expériences sont volontairement légères afin de rester exécutables sur CPU. Cela limite la profondeur des architectures et le volume des données utilisées. Les conclusions doivent donc être comprises comme une analyse expérimentale pédagogique plutôt que comme une recherche de performance maximale.

Pour améliorer le projet, on pourrait ajouter une recherche d'hyperparamètres plus systématique, une validation croisée pour les données tabulaires, une régularisation plus avancée pour les CNN, et un corpus parallèle réel plus grand pour la partie Seq2Seq.